

ADAPTIVE NEURO-FUZZY SYSTEMS

Many engineering problems can be solved by function approximation. We discuss now the adaptive neuro-fuzzy technics for function approximation:

- Subtractive clustering (fuzzy + tuning)
- Function approximation with multilayer neural networks
- Adaptive networks
- ANFIS (Adaptive Neuro-Fuzzy Inference System)
- ANFIS identification of nonlinear system dynamics.

1. Subtractive clustering

1.1. The problem statement

For the sake of simplicity, SISO system is considered (which can be generalized).

The form of a rule: $R_i : \text{if } x \text{ is } A_i \text{ then } y \text{ is } B_i, \quad i = 1, \dots, m.$

Questions, issues:

- What is the value of m , i.e. how many rules should be chosen?
- Which are the centers of the fuzzy sets A_i and B_i ?
- Which are the membership functions $\mu_{A_i}(x)$ and $\mu_{B_i}(y)$?

Assumptions:

- Teaching data: $(x_i, y_i), i = 1, \dots, N$
 - Options for the center of fuzzy sets:
 - centers chosen from data points (teaching points preferred) or
 - centers chosen on raster/grid line (interpolation preferred) $\rightarrow$ Following this way
- Rasters: rectangle (hypercube) $N_{ij} = (x_i^*, y_j^*)$.

1.2. Algorithm (Subtractive clustering)

1. Quantisation of x and y to the allowed raster centers, that is the choice of the raster $N_{ij} = (x_i^*, y_j^*)$ as intersections of gridlines.
- 2.
3. Approximation of the density of the sample points. A potential (mountain) function $M : N_{ij} \rightarrow R^1$ will be determined for this purpose by the following rule :

$$d(N_{ij}, (x_k, y_k)) = (x_k - x_i^*)^2 + (y_k - y_j^*)^2,$$

$$M(N_{ij}) = \sum_{k=1}^N \exp\{-\alpha \cdot d(N_{ij}, (x_k, y_k))\}.$$

$M(N_{ij})$ takes its maximum at gridpoint (i, j)

4. Initialization: $m := 1$, $M_1 := M$, and choice of α , β and the stop condition $\delta > 0$ ($M_{m+1}^* < \delta$)
5. Cycle:

- i. $M_m^* := \max_{ij} M_m(N_{ij})$, $N_m^* := \arg \max_{x_i^*, y_j^*} M_m(N_{ij}) = (\bar{x}_m^*, \bar{y}_m^*)$

(the maximum value of potential function and its location)

Potential function supervision (subtracting)

$$M_{m+1}(N_{ij}) := M_m(N_{ij}) - M_m^* \exp\{-\beta \cdot d(N_m^*, N_{ij})\}$$

- ii. Jump to step 4 if $M_{m+1}^* \geq \delta$, otherwise Stop.

1.3. The constructed fuzzy system

Cluster centers: $N_i^* = (\bar{x}_i^*, \bar{y}_i^*), i = 1, \dots, m$

The fuzzy relation for every cluster center $N_i^* = (\bar{x}_i^*, \bar{y}_i^*), i = 1, \dots, m$:

$$R_i : \text{if } x \text{ is near } x_i^* \text{ then } y \text{ is near } y_j^*, \quad i = 1, \dots, m.$$

Let the centers of the fuzzy sets A_i : $\bar{x}_i := \bar{x}_i^*$

Let the centers of the fuzzy sets B_i : $\hat{y}_i := \bar{y}_i^*$ (Sugeno system, hence this will be the firing value of the rule)

Gaussian membership functions for A_i (characterizing 'near'):

$$\mu_{A_i}(x) = \exp\left\{-\frac{1}{2}\left(\frac{x - \bar{x}_i}{\sigma_i}\right)^2\right\} =: \tau_i \quad (\text{The firing weight for rule consequent } \hat{y}_i)$$

Approximated output:

$$\hat{y} = \hat{f}(x) = \frac{\sum_{i=1}^m \tau_i \hat{y}_i}{\sum_{i=1}^m \tau_i} = \sum_{i=1}^m \tau_i^* \hat{y}_i$$

where

$$\tau_i^* = \frac{\tau_i}{\sum_{k=1}^m \tau_k}$$

MATLAB implementation: `genfis2`

1.4. The improvement of fuzzy system

The parameters $\bar{x}_i, \sigma_i$ and $\hat{y}_i$ can be further tuned by optimum seeking methods based on the available data: Gradient methods for optimum seeking (Strictly speaking, it is already not the part of subtractive clustering)

The partial error: $E_k = \frac{1}{2} [y_k - \hat{y}(x_k)]^2$ Total error: $E_{\text{total}} = \sum_{k=1}^N E_k$	If p is a parameter to be tuned then $\frac{\partial E_k}{\partial p} = -[y_k - \hat{y}(x_k)] \frac{\partial \hat{y}(x_k)}{\partial p}$ $\frac{\partial E_{\text{total}}}{\partial p} = \sum_{k=1}^N \frac{\partial E_k}{\partial p}$
The partial derivatives needed to the gradient vector: $\frac{\partial \hat{y}(x_k)}{\partial \hat{y}_i} = \tau_i^*$ $\frac{\partial \hat{y}(x_k)}{\partial \bar{x}_i} = \hat{y}_i \frac{\partial \tau_i}{\partial \bar{x}_i} \frac{1}{\sum_k \tau_k} + \sum_i \hat{y}_i \tau_i (-1) \frac{1}{(\sum_k \tau_k)^2} \frac{\partial \tau_i}{\partial \bar{x}_i} =$ $= \hat{y}_i \frac{\partial \tau_i}{\partial \bar{x}_i} \frac{1}{\sum_k \tau_k} - \hat{y}(x_k) \frac{1}{\sum_k \tau_k} \frac{\partial \tau_i}{\partial \bar{x}_i}$ $\frac{\partial \tau_i}{\partial \bar{x}_i} = \exp \left[-\frac{1}{2} \left(\frac{x_k - \bar{x}_i}{\sigma_i} \right)^2 \right] \left(-\frac{1}{2} \right) \frac{1}{\sigma_i^2} 2(x_k - \bar{x}_i)(-1) = \tau_i \frac{x_k - \bar{x}_i}{\sigma_i^2}$ $\frac{\partial \hat{y}(x_k)}{\partial \bar{x}_i} = [\hat{y}_i - \hat{y}(x_k)] \tau_i^* \frac{x_k - \bar{x}_i}{\sigma_i^2}$ $\frac{\partial \hat{y}(x_k)}{\partial \sigma_i} = [\hat{y}_i - \hat{y}(x_k)] \tau_i^* \frac{(x_k - \bar{x}_i)^2}{\sigma_i^3}$	Tuning based on $e = y_k - \hat{y}(x_k)$: $\hat{y}_i := \hat{y}_i + \gamma e \tau_i^*$ $\bar{x}_i := \bar{x}_i + \gamma e [\hat{y}_i - \hat{y}(x_k)] \tau_i^* (x_k - \bar{x}_i) / \sigma_i^2$ $\sigma_i := \sigma_i + \gamma e [\hat{y}_i - \hat{y}(x_k)] \tau_i^* (x_k - \bar{x}_i)^2 / \sigma_i^3$

2. Function approximation by multilevel neural network

Elementary neuron

Unknown: $y = f(x)$

Application: FFNN (Feedforward Neural Network) (MIMO case)

Static elementary neuron:

$$\hat{y} = \sigma \left(\sum_{i=1}^n w_i x_i + t \right)$$

w_i weighting factor

t offset (threshold)- constant 1 fictitious input

$\sigma(\cdot)$ transition függvény

Teaching the neuron $\equiv$ Choose the appropriate weights w_i

Examples for transition functions

Functions

$$\sigma(x) = \frac{1}{1 + \exp(-cx)} \in [0, 1]$$

$$\sigma(x) = \frac{2}{1 + \exp(-cx)} - 1 \in [-1, 1]$$

$$\sigma(x) = \tanh(cx) = \frac{\exp(cx) - \exp(-cx)}{\exp(cx) + \exp(-cx)} \in [-1, 1]$$

$$\sigma(x) = cx \in [-\infty, \infty]$$

and derivatives:

logsig $\sigma'(x) = \frac{c \exp(-cx)}{[1 + \exp(-cx)]^2} = c \sigma(x)(1 - \sigma(x))$

bipoláris $\sigma'(x) = \frac{2c \exp(-cx)}{[1 + \exp(-cx)]^2} = \frac{c}{2} [1 + \sigma(x)] \cdot [1 - \sigma(x)]$

tansig $\sigma'(x) = c[1 - \tanh^2(cx)] = c[1 + \sigma(x)] \cdot [1 - \sigma(x)]$

purelin $\sigma'(x) = c$

Single layer neural network

Number of inputs: n_i
outputs: n_o

Outputs:

$$\hat{y}_p = \sigma_p \left(\sum_{j=1}^{n_i} w_{pj} x_j \right), \quad p = 1, 2, \dots, n_o$$

or

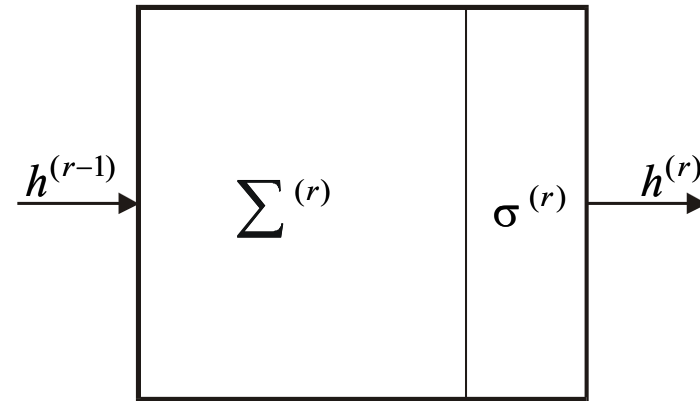
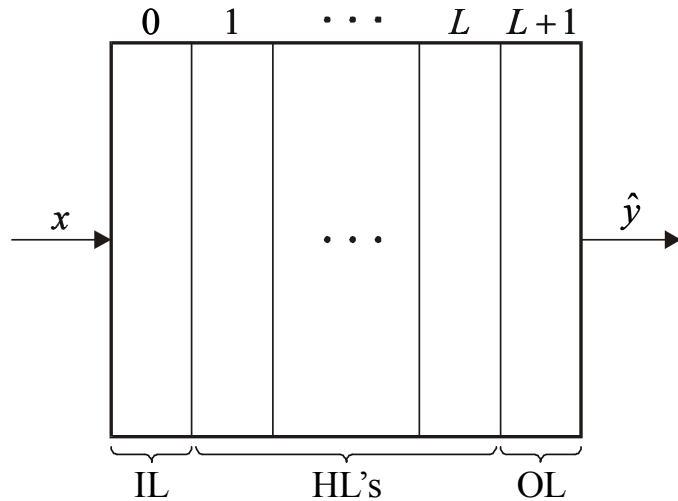
$$\hat{y}_p^* = \sum_{j=1}^{n_i} w_{pj} x_j, \quad \hat{y}_p = \sigma_p(\hat{y}_p^*), \quad p = 1, 2, \dots, n_o$$

$\hat{y}_p^*$ net input (the resulting weighted input)

Transition function is typically the same for every neuron: $\sigma_p(\cdot) := \sigma(\cdot)$

Multilayer neural network (FFNN type)

Consists of an input layer (IL), one or more hidden layers (HL's) and an output layer (OL). The layers are numbered: the 0-th input layer, the $(L+1)$ -th output layers, and the intermediate hidden layers:



Let

$h^{(r-1)}$ the input of the (r) -th layer
 $h^{(r)}$ the output of the (r) -th layer
 $n_o^{(r)}$ The number of neurons in the (r) -th layer

Then

Net inputs (the resulting weighted inputs) and the outputs of the layer:

In compact form:

$$\Sigma^{(r)} : h_p^{*(r)} = \sum_j w_{pj}^{(r)} h_j^{(r-1)}, p = 1, 2, \dots, n_o^{(r)}$$

$$\sigma^{(r)} : \sigma_p^{(r)} = \sigma_p^{(r)}(h_p^{*(r)}), p = 1, 2, \dots, n_o^{(r)}$$

$W^{(r)}$ Weighting matrix of the (r) -th layer

$h^{(0)} = x$ input of the neural network

$h^{(L+1)} = \hat{y}$ output of the neural network

$$h_p^{*(r)} = \sum_{j=1}^{n_o^{(r-1)}} w_{pj}^{(r)} h_j^{(r-1)}, \quad h_p^{(r)} = \sigma_p^{(r)}(h_p^{*(r)})$$

Forward propagation in the neural network

The input of the NN:

$$x = (x_1, x_2, \dots, x_{n_i})^T$$

Forward propagation of a signal in the layers:

$$x = h^{(0)} \xrightarrow{w^{(1)}, \sigma^{(1)}} h^{*(1)}, h^{(1)} \dots h^{*(L)}, h^{(L)} \xrightarrow{w^{(L+1)}, \sigma^{(L+1)}} h^{*(L+1)}, h^{(L+1)} = \hat{y} \quad (\text{FP})$$

Output signal at the output of NN:

$$\hat{y} = (\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{n_o})^T$$

Goal: Reach a prescribed behavior (function) based on teaching points

Teaching samples: $\{x(k), y(k)\}_{k=1}^N$

The response of NN: $\{x(k), \hat{y}(k)\}_{k=1}^N$

Problem to be solved: Minimize error criterion on the base of weights $w_{pj}^{(r)}$.

Method: compute the partial derivatives of the error criterion by the weights $w_{pj}^{(r)}$,
 $\rightarrow$ Optimization by gradient, conjugate gradient or gradient-like method.

Error backpropagation (BP) algorithms

Let $\Phi : \mathbb{R}^1 \rightarrow [0, \infty)$ be a positive definite, convex and (almost everywhere) differentiable function, for example $\Phi(e) = \frac{1}{2}e^2$, then

A partial error $E(k)$ of the NN at input $x(k)$

$$E(k) = \sum_{i=1}^{n_o} \Phi(y_i(k) - \hat{y}_i(k))$$

The total error of NN for every inputs:

$$E_{\text{total}} = \sum_{k=1}^N E(k)$$

It is known that

$$\frac{\partial E_{\text{total}}}{\partial w_{pj}^{(r)}} = \sum_{k=1}^N \frac{\partial E(k)}{\partial w_{pj}^{(r)}}$$

Key problem:

$$\frac{\partial E(k)}{\partial w_{pj}^{(r)}} \text{ to be obtained}$$

omitting index k :

$$\begin{aligned} w_{pj}^{(r)} \rightarrow h_p^{*(r)} &= \sum_j w_{pj}^{(r)} h_j^{(r-1)} \\ h_p^{(r)} &= \sigma_p^{(r)}(h_p^{*(r)}) \Rightarrow \\ h_l^{*(r+1)} &= \sum_p w_{lp}^{(r+1)} h_p^{(r)}. \end{aligned}$$

Based on the chain rule of differentiation:

$$\begin{aligned} \frac{\partial E}{\partial w_{pj}^{(r)}} &= \frac{\partial E}{\partial h_p^{*(r)}} \cdot \frac{\partial h_p^{*(r)}}{w_{pj}^{(r)}} = \frac{\partial E}{\partial h_p^{*(r)}} h_j^{(r-1)} \\ \frac{\partial E}{\partial h_p^{*(r)}} &= \sum_l \underbrace{\frac{\partial E}{\partial h_l^{*(r+1)}}}_{\varepsilon_l^{(r+1)}} \cdot \underbrace{\frac{\partial h_l^{*(r+1)}}{\partial h_p^{(r)}}}_{w_{lp}^{(r+1)}} \cdot \underbrace{\frac{\partial h_p^{(r)}}{\partial h_p^{*(r)}}}_{\sigma_p'^{(r)}(h_p^{*(r)})}, \quad (\text{EBP}) \\ \varepsilon_p^{(r)} &= \sigma_p'^{(r)}(h_p^{*(r)}) \sum_l \varepsilon_l^{(r+1)} w_{lp}^{(r+1)}. \end{aligned}$$

Teaching multilayer FFNN

Algorithms [Teaching FFNN with error backpropagation]

1. Evaluate output with Forward propagation of input:

$$x = h^{(0)} \xrightarrow{W^{(1)}, \sigma^{(1)}} h^{*(1)}, h^{(1)} \dots h^{*(L)}, h^{(L)} \xrightarrow{W^{(L+1)}, \sigma^{(L+1)}} h^{*(L+1)}, h^{(L+1)} = \hat{y}$$

2. Error backpropagation:

2.1. Obtain error: $e = y - \hat{y}$

2.2. Obtain weights by using the gradients of weights in backpropagation

Start: $r = L + 1$

$$\varepsilon_p^{(L+1)} = \frac{\partial E}{\partial h_p^{*(L+1)}} = -\Phi'(y_p - \hat{y}_p) \sigma_p'^{(L+1)}(h_p^{*(L+1)}),$$

$$\frac{\partial E}{\partial w_{pj}^{(L+1)}} = \varepsilon_p^{(L+1)} h_j^{(L)}.$$

Backward recursion: $r = L, L - 1, \dots, 1$

$$\varepsilon_p^{(r)} = \sigma_p'^{(r)}(h_p^{*(r)}) \sum_l \varepsilon_l^{(r+1)} w_{lp}^{(r+1)},$$

$$\frac{\partial E}{\partial w_{pj}^{(r)}} = \varepsilon_p^{(r)} h_j^{(r-1)}.$$

Options for tuning:

➤ Sequentially (on-line) after every presented input

➤ Batch mode (off-line), after every input is presented

(It is better since E_{total} is used + effective [e.g. conjugate] gradient technics can be used)

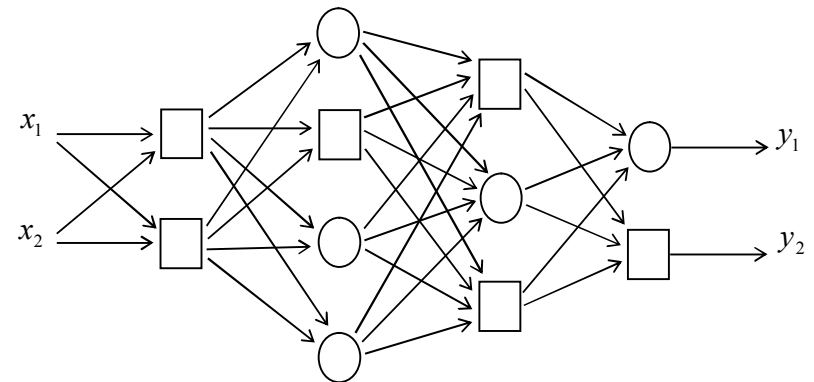
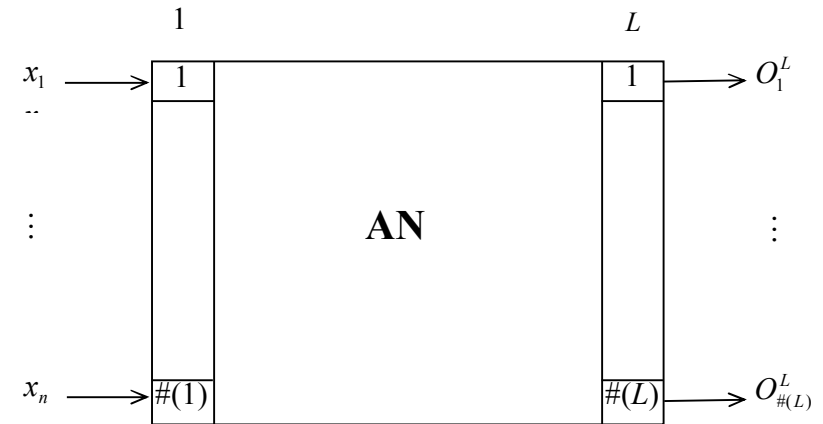
3. Adaptive networks (AN)

3.1. The structure of adaptive networks

ANs are some kind of generalization of NNs and fuzzy systems.

Properties:

- consists of nodes connected by directed edges.
- Signals spread forward along the edge
- organized in (L) layers
- does not contain any feedback
- Input: x_i , Output: O_i^L
- Number of nodes in the k th layer: $\#(k)$
- **Similar to FFNN, but there are no weights on the edges!**
- The function of nodes:
 $O_i^k = O_i^k(O_1^{k-1}, \dots, O_{\#(k-1)}^{k-1}, a, b, c, \dots)$
 $O_1^{k-1}, \dots, O_{\#(k-1)}^{k-1}$ are the input parameters, $a, b, c, \dots$ are the function parameters.
- Set of AN parameters: S
- Notation of adaptive nodes: rectangle.
- Notation of fix nodes: circle.



3.2 The error of adaptive networks

The target set of input/output vectors: $\{x(k), y(k)\}_{k=1}^N$

The output vector of AN: $\hat{y} = (O_1^L, \dots, O_{\#(L)}^L)^T$

Gial: Minimize total error $E_{\text{total}} = \sum_{k=1}^N E(k)$ by teaching where

$$E(k) = \frac{1}{2} \sum_{i=1}^{\#(L)} (y_i(k) - \hat{y}_i(k))^2 = \frac{1}{2} \sum_{i=1}^{\#(L)} (y_i(k) - O_i^L(k))^2$$

The parameters of the adaptive network can be optimized for example by the ordinary gradient method where $\alpha \in \mathcal{S}$ is any parameter:

$$\Delta\alpha = -\eta \frac{\partial E_{\text{total}}}{\partial \alpha} = -\frac{\lambda}{\sqrt{\sum_{\alpha} \left| \frac{\partial E_{\text{total}}}{\partial \alpha} \right|^2}} \frac{\partial E_{\text{total}}}{\partial \alpha}$$

where

λ is the step length

$$\frac{\partial E_{\text{total}}}{\partial \alpha} = \sum_{k=1}^N \frac{\partial E(k)}{\partial \alpha}$$

$$\frac{\partial E(k)}{\partial \alpha} = \sum_{O \in O_{\alpha}^*} \frac{\partial E(k)}{\partial O} \cdot \frac{\partial O}{\partial \alpha}$$

(O_{α}^* : nodes depend on α)

$O(\cdot)$ analytically known

$\rightarrow \partial O / \partial \alpha$ is computable $\Rightarrow$

Computation of $\partial E(k) / \partial O$ with backward recursion:

$$\frac{\partial E}{\partial O_i^L} = -(y_i - O_i^L) = -(y - \hat{y}_i) \quad (\text{last layer})$$

$$\frac{\partial E(k)}{\partial O_i^l} = \sum_{m=1}^{\#(l+1)} \frac{\partial E(k)}{\partial O_m^{l+1}} \cdot \frac{\partial O_m^{l+1}}{\partial O_i^l}$$

(since O_i^l is the input of

$$O_i^{l+1} = O_i^{l+1}(O_1^l, \dots, O_{\#(l)}^l, \dots)$$

4. ANFIS

(Adaptive Neuro-Fuzzy Inference System or Adaptive Network Based Fuzzy Inference System)

Sugeno-type single output fuzzy systems can be easily converted to adaptive networks (AN).

First order Sugeno fuzzy system:

$$R_i : \text{if } x_1 \text{ is } A_1^i \text{ and } \dots \text{and } x_n \text{ is } A_n^i \text{ then } y \text{ is } c_{i1}x_1 + \dots c_{in}x_n + c_{i0}, \\ i = 1, \dots, m.$$

(Zero order Sugeno fuzzy system: $c_{i1} = \dots = c_{in} = 0$)

Theorem [Li Xin Wang] Any continuous function can be approximated (over a compact set) by *zero order Sugeno system* for any prescribed precision $\varepsilon > 0$, when Gaussian membership functions are used.

Remarks:

- For decreasing ε the number of necessary relations will increase.
- First order Sugeno system is more general than the zero order one $\rightarrow$ We use this one

The form of membership function:

$$\mu_{A_j^i}(x_j) := \exp \left[-\frac{1}{2} \left(\frac{x_j - \bar{x}_j^i}{\sigma_j^i} \right)^2 \right]$$

The steps of approximation for input $x = (x_1, \dots, x_n)^T$ using Sugeno system (with product as T-norm):

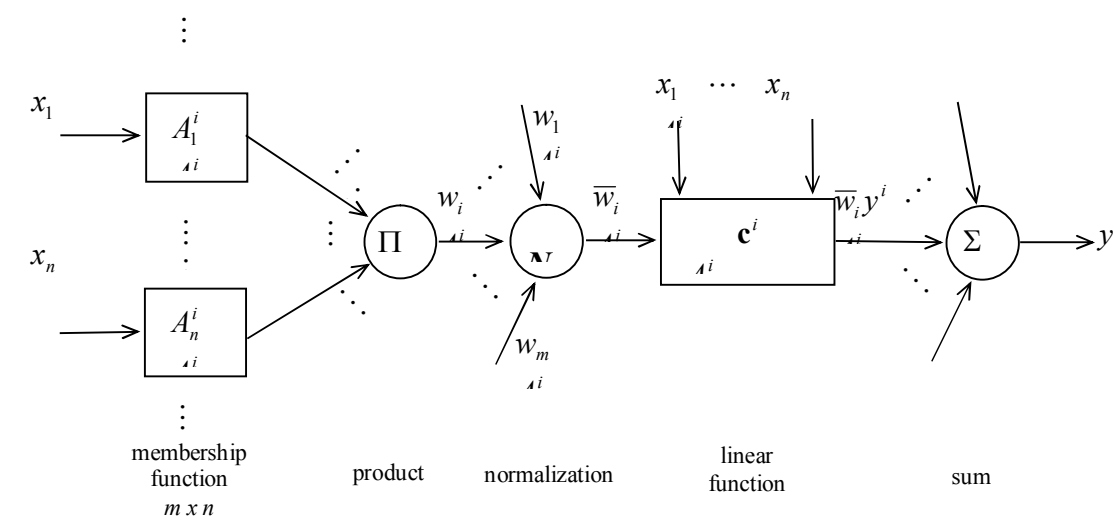
$$w_i := \mu_{A_1^i}(x_1) \cdots \mu_{A_n^i}(x_n)$$

$$\hat{y}^i(x) := c_{i1}x_1 + \cdots + c_{in}x_n + c_{i0}$$

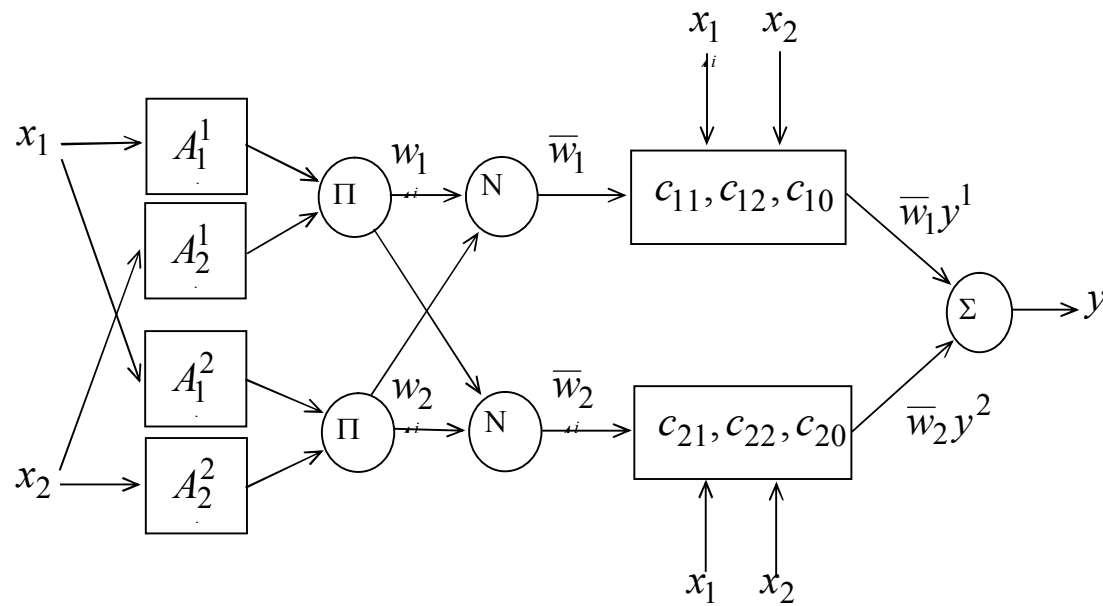
$$\hat{y} = \hat{f}(x) = \frac{\sum_{i=1}^m w_i \hat{y}^i(x)}{\sum_{i=1}^m w_i} = \sum_{i=1}^m \bar{w}_i \hat{y}^i(x)$$

The parameters of first order Sugeno system: $\bar{x}_j^i, \sigma_j^i, c_{i1}, \dots, c_{in}, c_{i0}, \quad j = 1, \dots, n; \quad i = 1, \dots, m$

Adaptive network part equivalent to the i-th relation of the Sugeno system:



Sugeno System having 2 inputs, 1 output, 2 relations in adaptive network form:



Teaching ANFIS:

The prescribed data (to be approximated): (x, y)

Approximated output by the first order Sugeno system: $\hat{y}(x)$

Computation of gradient vector:

Substituting the specific parameters into general parameter p , the partial derivatives are:

$$E(k) = \frac{1}{2} [y(k) - \hat{y}(x(k))]^2$$

$$\frac{\partial \hat{y}(x)}{\partial c_{ij}} = \begin{cases} x_j \bar{w}_i, & \text{if } j = 1, \dots, n \\ \bar{w}_i, & \text{if } j = 0, \end{cases}$$

$$E_{\text{total}} = \sum_k E(k)$$

$$\frac{\partial \hat{y}(x)}{\partial \bar{x}_j^i} = [\hat{y}^i(x) - y(x)] \bar{w}_i \frac{x_j - \bar{x}_j^i}{(\sigma_j^i)^2}$$

$$\frac{\partial E(k)}{\partial p} = -[y(k) - \hat{y}(x(k))] \frac{\partial \hat{y}(x(k))}{\partial p}$$

$$\frac{\partial \hat{y}(x)}{\partial \sigma_j^i} = [\hat{y}^i(x) - y(x)] \bar{w}_i \frac{(x_j - \bar{x}_j^i)^2}{(\sigma_j^i)^3}$$

$$\frac{\partial E_{\text{total}}}{\partial p} = \sum_k \frac{\partial E(k)}{\partial p}$$

Any type of optimization methods based on the gradient can be applied

The parameters of the consequent parts appear linear in $\hat{y}(x)$:

$$\hat{y}(x) = \varphi^T(x) \mathcal{G} := \begin{bmatrix} \bar{w}_1(x^T \ 1) & \bar{w}_2(x^T \ 1) \end{bmatrix} \begin{bmatrix} c^1 \\ c^2 \end{bmatrix} = \begin{pmatrix} \bar{w}_1 x_1 & \bar{w}_1 x_2 & \bar{w}_1 & \bar{w}_2 x_1 & \bar{w}_2 x_2 & \bar{w}_2 \end{pmatrix} \begin{pmatrix} c_{11} \\ c_{12} \\ c_{10} \\ c_{21} \\ c_{22} \\ c_{20} \end{pmatrix}.$$

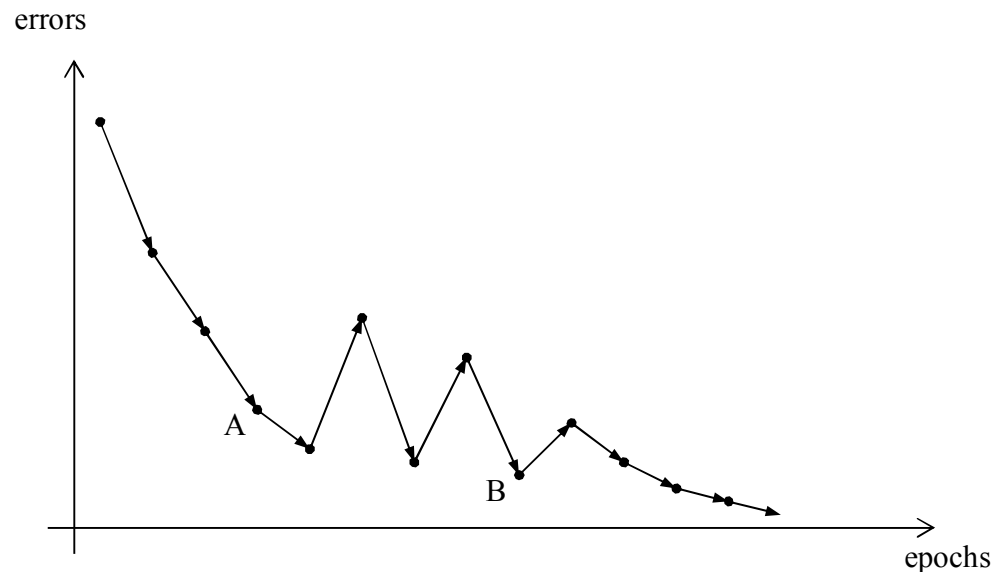
The optimization of the first order Sugeno fuzzy system can be performed by the *hybrid* ANFIS optimization technique.

The following are performed in each teaching step (epoch):

1. The parameters of the consequent parts will be optimized by the robust and quick one-step LS technique without using the gradient.
2. The remaining parameters will be optimized by gradient search.

remarks:

- The use of LS method for the consequent part parameters enormly increases the convergence speed
- The step length of the ordinary gradient search can be adaptively modified for further improvement of the convergence of the antecedent parameters E.g.:
 - a. Increase stepsize after 4 downs (A)
 - b. Decrase step size after 2 combinations of 1 up and 1 down (B)



Teaching of multi-output system

Concept: Teaching for each output separately

Simplification:

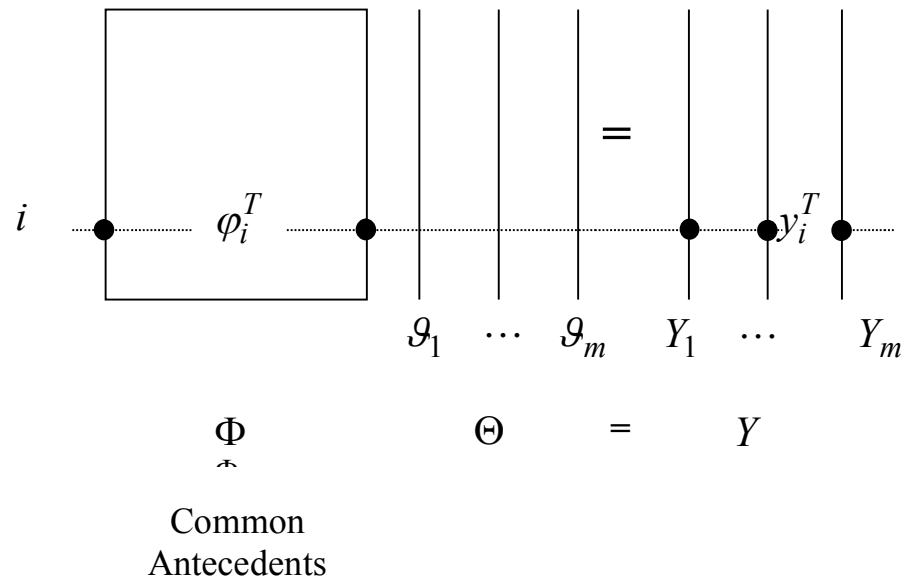
if the *antecedents are common* for every output (R_i is the same for every output) then:

$$Y_j = \Phi \mathcal{G}_j, \quad j = 1, \dots, r, \quad r = \dim y \Rightarrow$$

$$\begin{bmatrix} Y_1 & \dots & Y_r \end{bmatrix} = \Phi \begin{bmatrix} \mathcal{G}_1 & \dots & \mathcal{G}_r \end{bmatrix}$$

$$Y = \Phi \Theta,$$

(Φ közös minden $\mathcal{G}_i$ paramétervektorra)



In this case Y and Θ are *matrices* (not vectors) which makes it possible to simplify the computation of the LS solution

Batch mode:
$$\hat{\Theta} = (\Phi^T \Phi)^{-1} \Phi^T Y$$

Recursive mode:
$$P_i = \frac{1}{\lambda} \left\{ P_{i-1} - P_{i-1} \varphi_i \frac{1}{\lambda + \varphi_i^T P_{i-1} \varphi_i} \varphi_i^T P_{i-1} \right\}$$

$$\hat{\Theta}_i = \hat{\Theta}_{i-1} + P_i \varphi_i (y_i^T - \varphi_i^T \hat{\Theta}_{i-1})$$

5. Nonlinear dynamical system identification using ANFIS

Problem: Approximate the nonnonlinear, discrete time system $y(t) = f(y(t-1), y(t-2), u(t-1), u(t-2))$

i.e. $y = f(x)$ unknown nonlinear function should be approximated where $x = (x_1, x_2, x_3, x_4)^T$ and

$$x_1(t) := y(t-1), x_2(t) := y(t-2), x_3(t) := u(t-1), x_4(t) := u(t-2)$$

Given: $\{u(t), y(t)\}_{t=1}^N$ input/output measurements (available data) of the system .

Concept:

1. Generate Sugeno fuzzy system with subtractive clustering (with genfis2 MATLAB function).

The rules of the generated fuzzy system:

$$R_i : \text{if } x_1 \text{ is } A_1^i \text{ and } \dots \text{and } x_n \text{ is } A_n^i \text{ then } y \text{ is } c_{i1}x_1 + \dots c_{in}x_n + c_{i0},$$

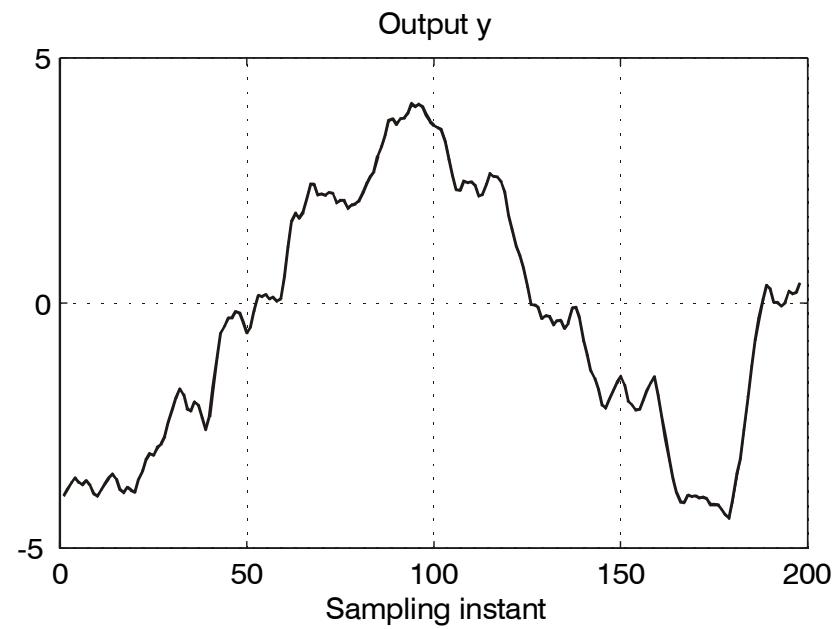
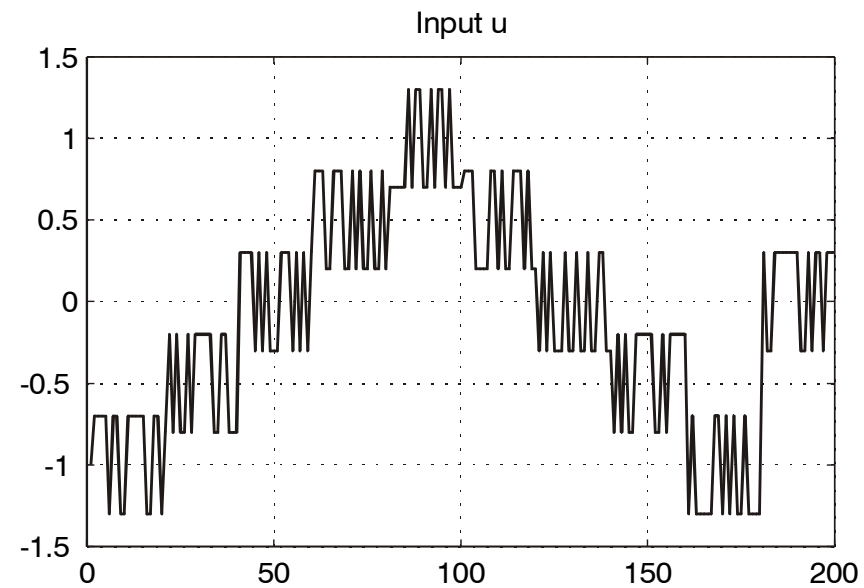
$$i = 1, \dots, m,$$

where $c_{i1} = \dots = c_{in} = 0$ (zero order fuzzy system)

2. Improvement with ANFIS-al (anfis MATLAB function)

Example:

Input and output:



Results:

- 3 rules:

$$R_1 : \text{if } x_1 \text{ is } \exp\left(-\frac{(x_1 + 1.589)^2}{2 \cdot 2.981^2}\right) \text{ and } x_2 \text{ is } \exp\left(-\frac{(x_2 + 1.744)^2}{2 \cdot 2.983^2}\right) \\ \text{and } x_3 \text{ is } \exp\left(-\frac{(x_3 + 0.2744)^2}{2 \cdot 0.9167^2}\right) \text{ and } x_4 \text{ is } \exp\left(-\frac{(x_4 + 0.2179)^2}{2 \cdot 0.8897^2}\right) \\ \text{then } \hat{y}_1 = 1.048x_1 - 0.2336x_2 + 0.4642x_3 + 0.1942x_4 - 0.02663,$$

$$R_2 : \text{if } x_1 \text{ is } \exp\left(-\frac{(x_1 - 2.435)^2}{2 \cdot 2.99^2}\right) \text{ and } x_2 \text{ is } \exp\left(-\frac{(x_2 - 2.261)^2}{2 \cdot 2.991^2}\right) \\ \text{and } x_3 \text{ is } \exp\left(-\frac{(x_3 - 0.7452)^2}{2 \cdot 0.872^2}\right) \text{ and } x_4 \text{ is } \exp\left(-\frac{(x_4 - 0.7353)^2}{2 \cdot 0.8807^2}\right) \\ \text{then } \hat{y}_1 = 1.028x_1 - 0.2044x_2 + 0.3663x_3 + 0.2104x_4 + 0.1375,$$

$$R_3 : \text{if } x_1 \text{ is } \exp\left(-\frac{(x_1 + 4.001)^2}{2 \cdot 2.981^2}\right) \text{ and } x_2 \text{ is } \exp\left(-\frac{(x_2 + 3.963)^2}{2 \cdot 2.981^2}\right) \\ \text{and } x_3 \text{ is } \exp\left(-\frac{(x_3 + 1.218)^2}{2 \cdot 1.053^2}\right) \text{ and } x_4 \text{ is } \exp\left(-\frac{(x_4 + 1.258)^2}{2 \cdot 1.003^2}\right) \\ \text{then } \hat{y}_1 = 1.057x_1 - 0.2306x_2 + 0.1677x_3 + 0.09548x_4 - 0.4582.$$

Performance of ANFIS:

